

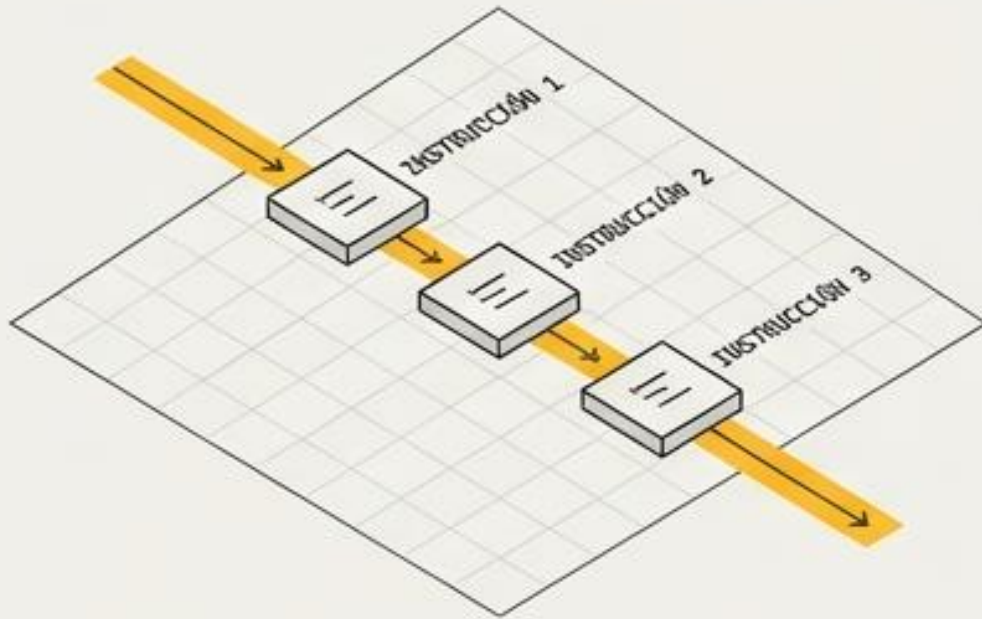


El Blueprint de la Lógica en Python

Guía visual de control de flujo: Condicionales, bucles y arquitecturas robustas.

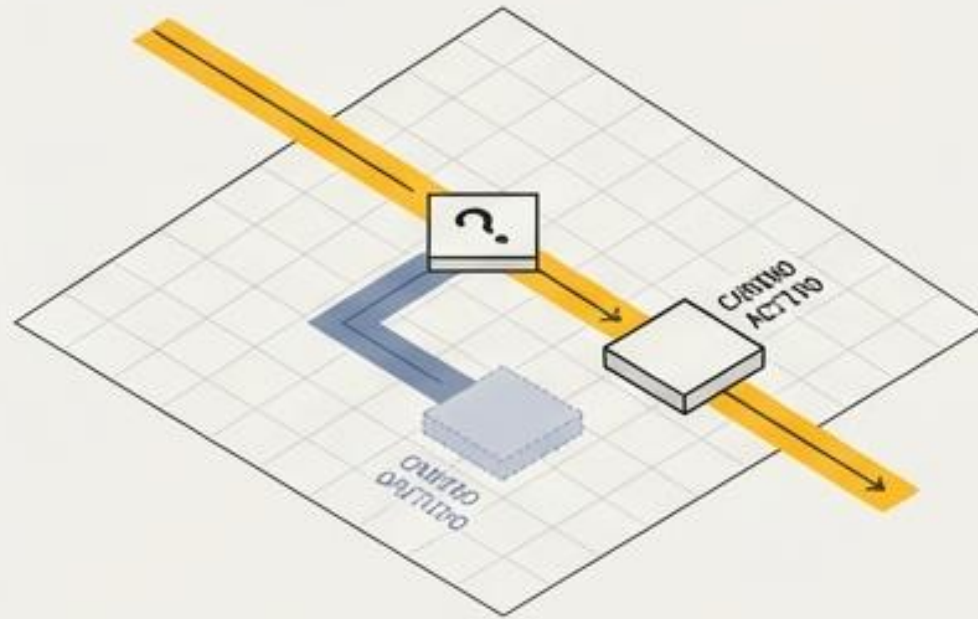
La Anatomía del Flujo de Control

Ejecución Lineal



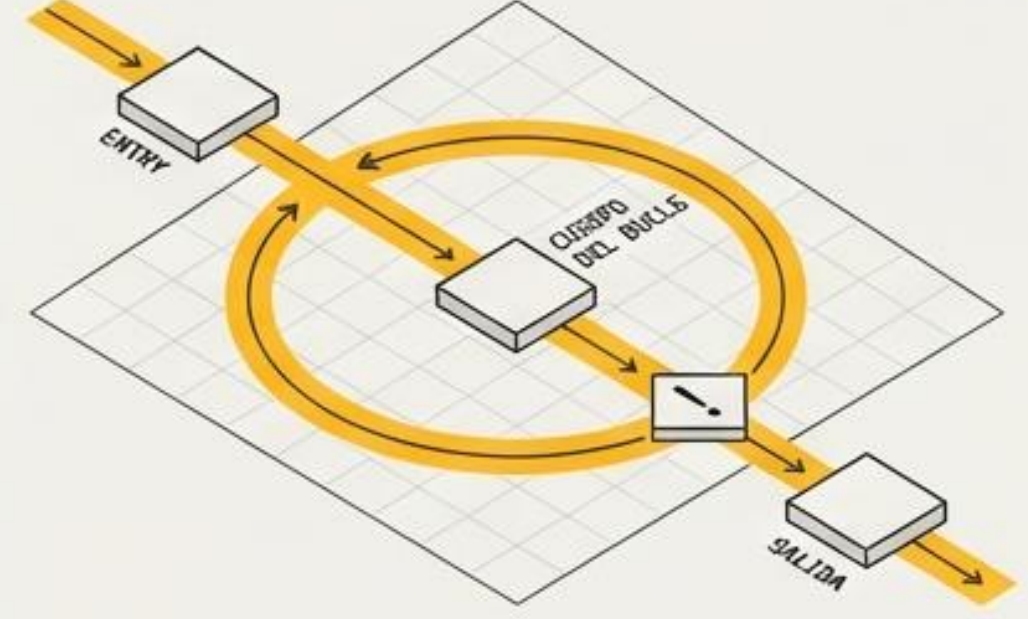
El código corre de arriba hacia abajo, sin desviarse.

Ramificación (Decisiones)



El programa evalúa una condición y elige un único camino.

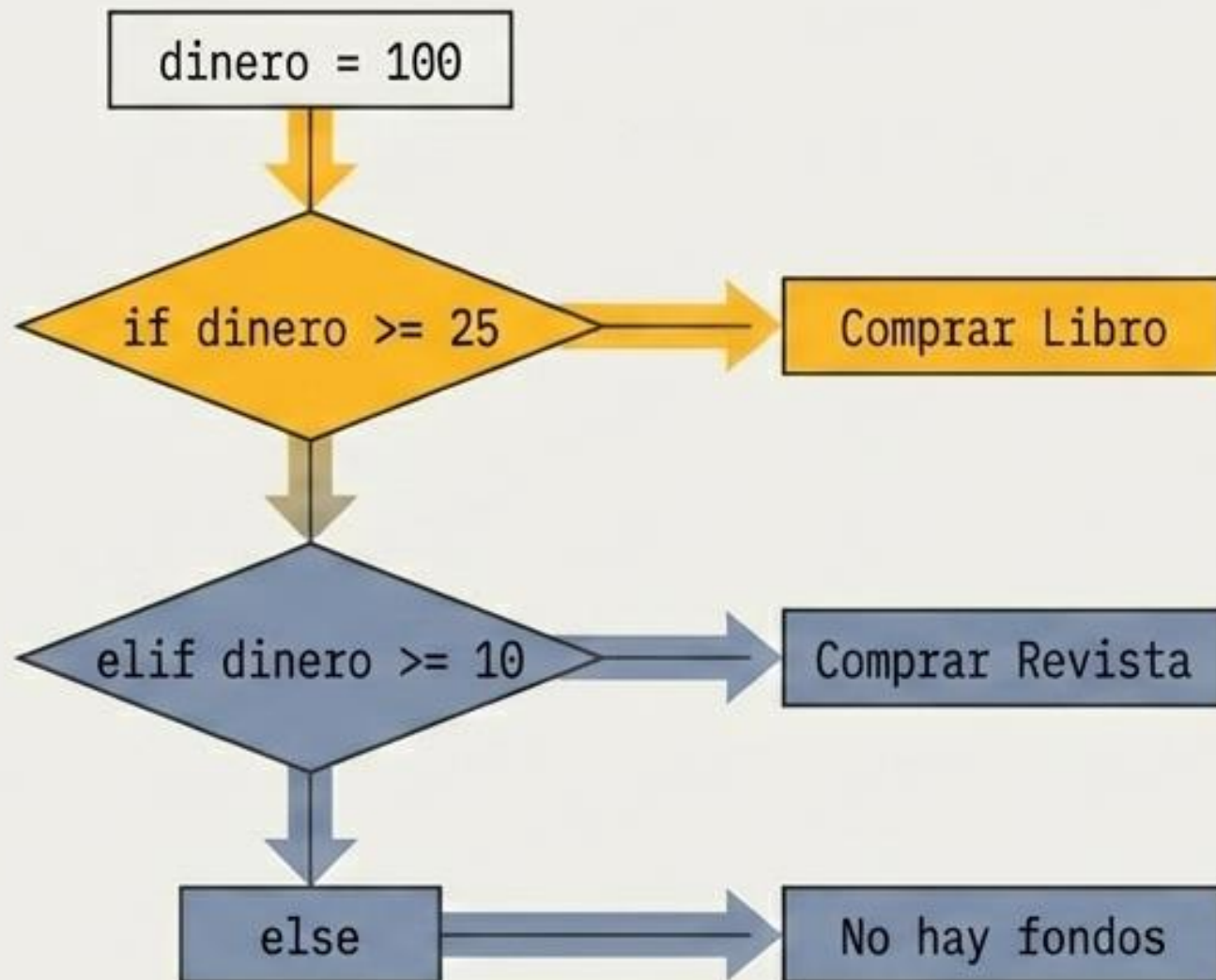
Iteración (Bucles)



Una sección se repite hasta que se cumple una condición de salida.

Decisiones Básicas: El Enrutador Lógico

Python evalúa las condiciones de arriba hacia abajo. En cuanto encuentra la primera evaluación True, ejecuta ese bloque y omite el resto de la estructura elif/else.

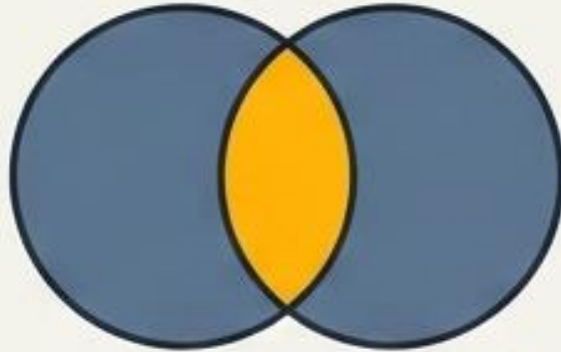


```
if dinero >= 25:  
    print('Comprar Libro')  
elif dinero >= 10:  
    print('Comprar Revista')  
else:  
    print('No hay fondos')
```

Operadores Lógicos y la Trampa de la Anidación

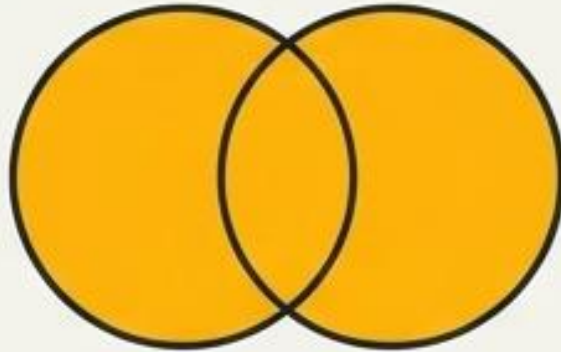
Combinación (and / or)

and



Ambas condiciones deben ser verdaderas

or



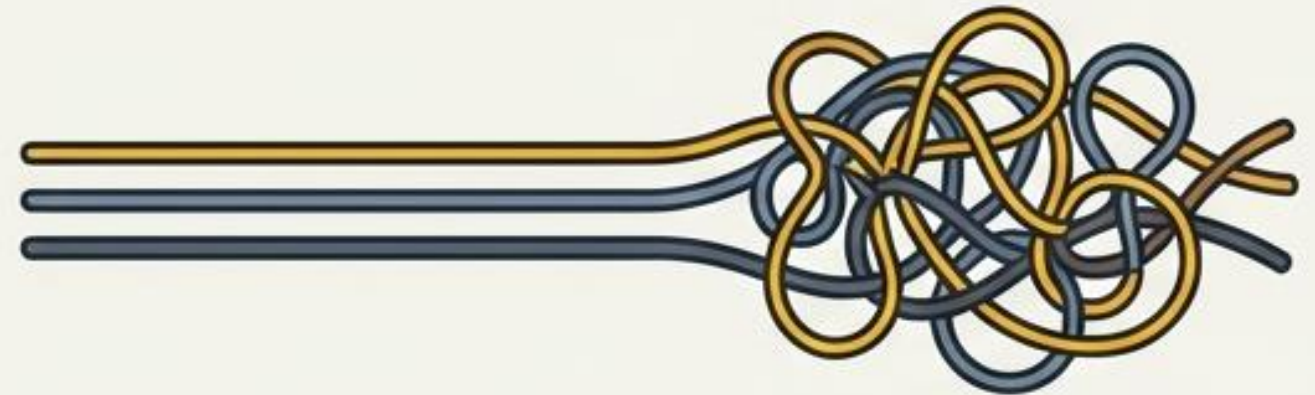
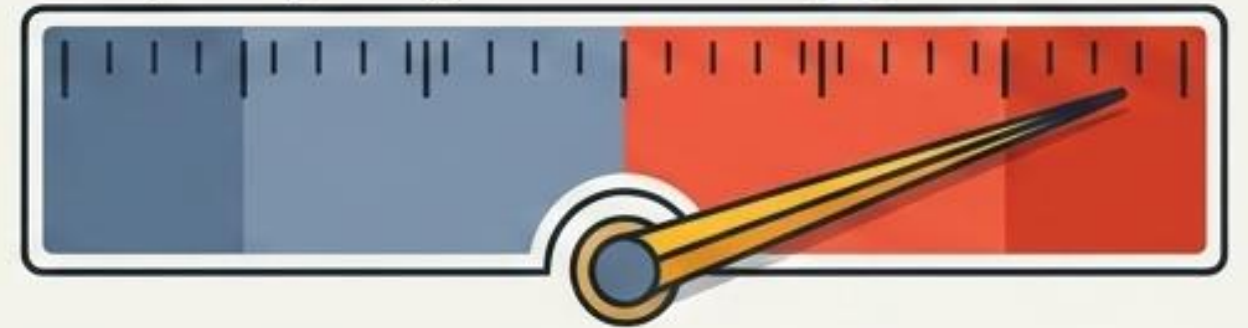
Al menos una debe ser verdadera

Utiliza and y or para evaluar múltiples requisitos en una sola línea de manera elegante.

Alerta Arquitectónica: Anidación

Complexity Gauge

⚠ ALTO RIESGO



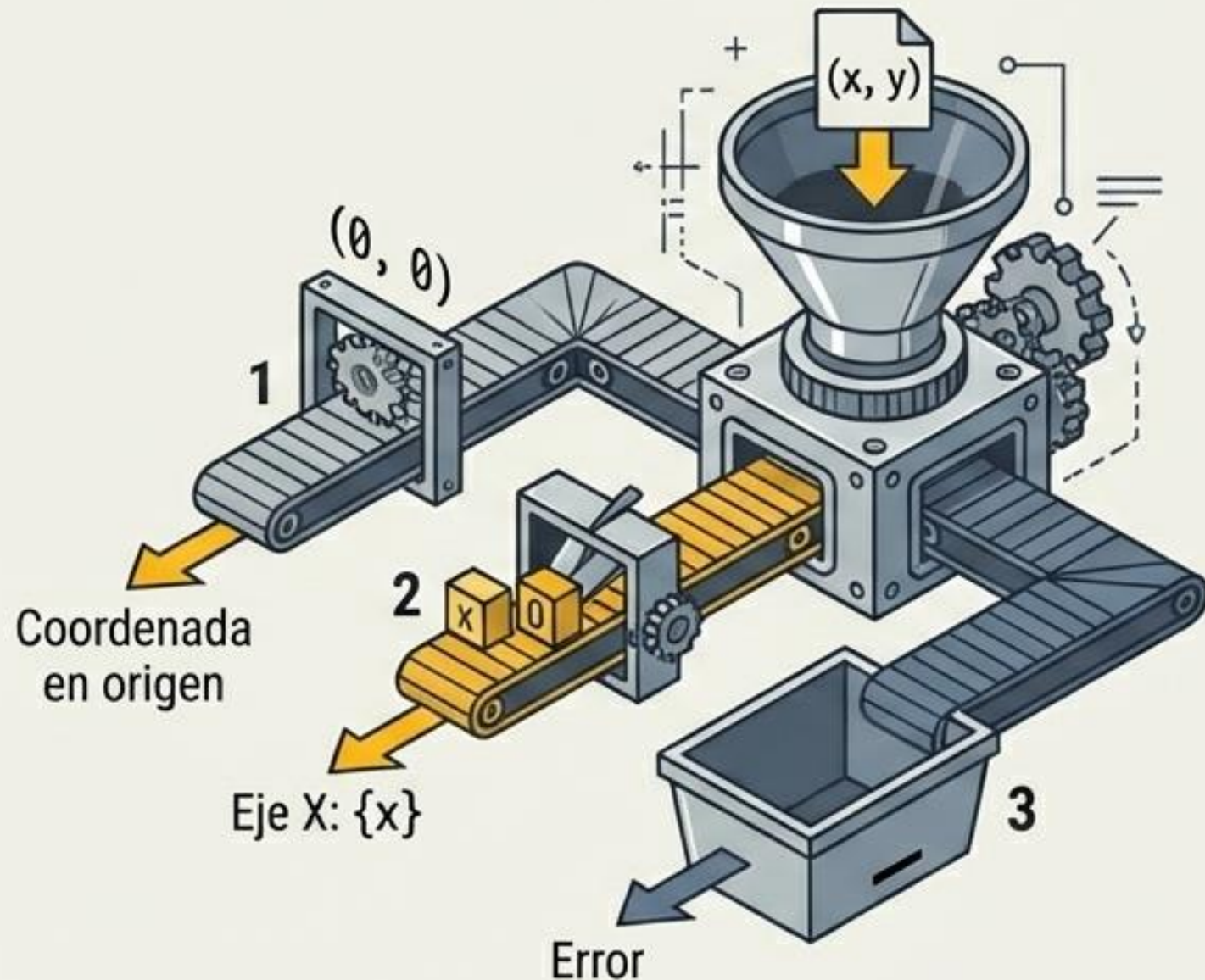
FLUJO LIMPIO

ESPAGUETI DE CÓDIGO

Cuidado: Anidar demasiados condicionales (un if dentro de otro if) destruye la legibilidad. Si superas los 3 niveles de profundidad, es momento de refactorizar.

Python Moderno: Structural Pattern Matching (match-case)

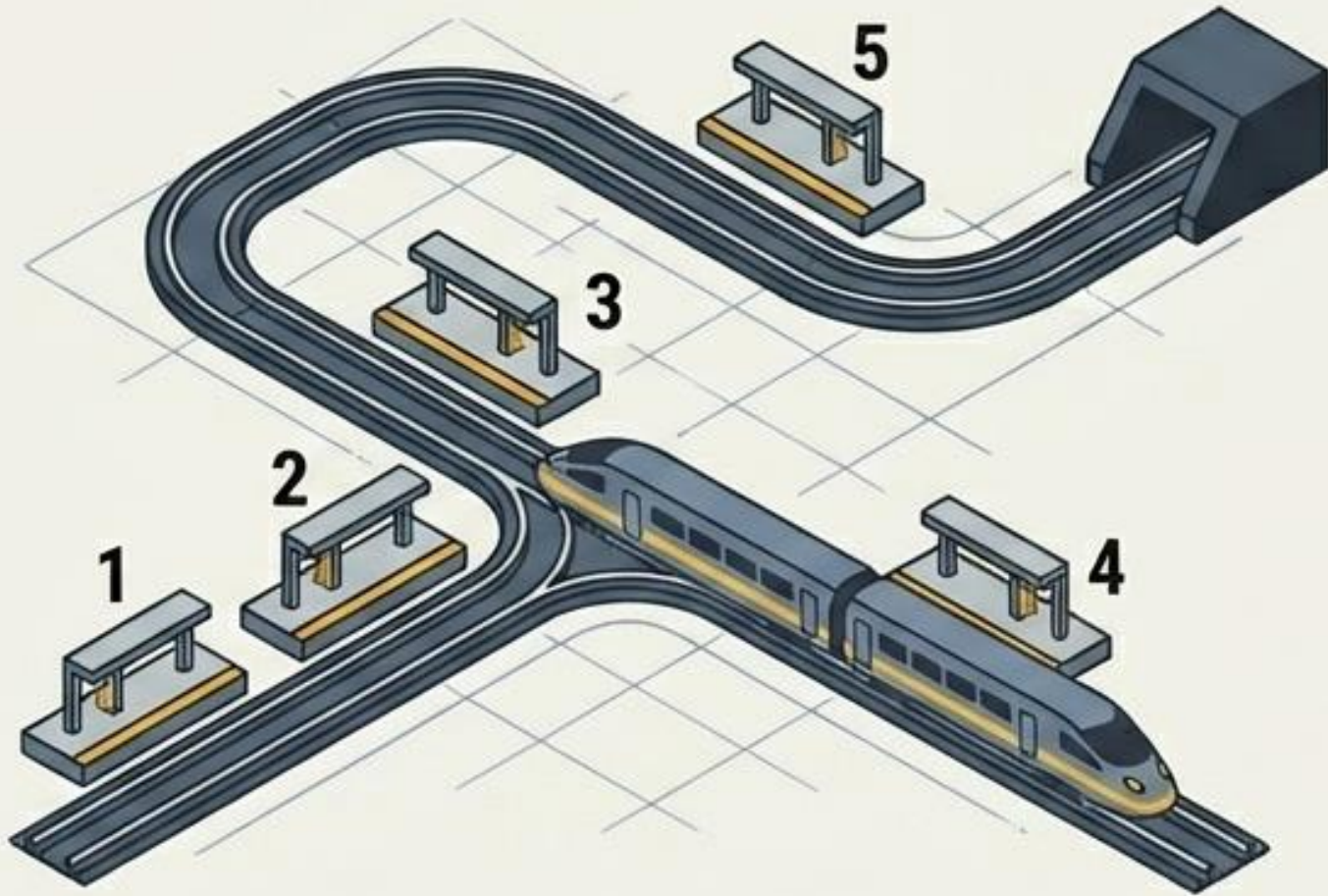
Introducido en Python 3.10, match no solo reemplaza cadenas largas de elif. Su verdadero poder reside en 'desempaquetar' estructuras de datos complejas directamente en el condicional.



```
match coordenada:
    case (0, 0):
        print('Coordenada en origen')
    case (x, 0):
        print(f'Eje X: {x}')
    case (0, y):
        print(f'Eje Y: {y}')
    case (x, y):
        print(f'Coordenada en: {x}, {y}')
    case _:
        print('Error')
```


Los Dos Motores de Iteración en Python

El Bucle for



Iteraciones conocidas.
Procesamiento de colecciones.
Actúa directamente sobre elementos.

El Bucle while

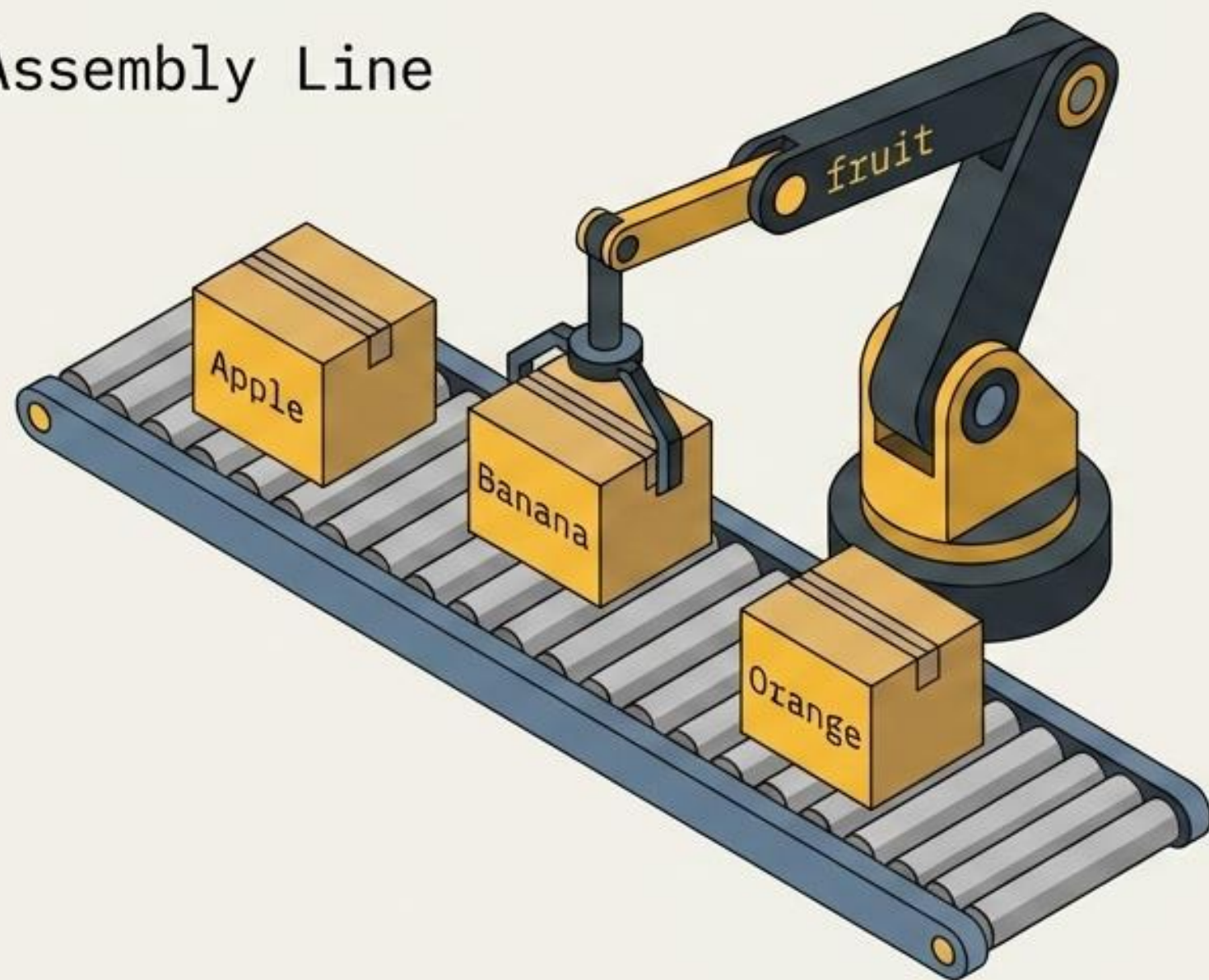


Iteraciones desconocidas.
Ejecución basada en estados.
Gira infinitamente hasta que la condición cambie a False.

El Bucle for: Dominando Colecciones

A diferencia de otros lenguajes, el `for` en Python no requiere contadores manuales. **Itera directamente** sobre los elementos de cualquier secuencia.

Assembly Line



Las 3 formas de `range()`

`range(stop)`



`range(start, stop)`



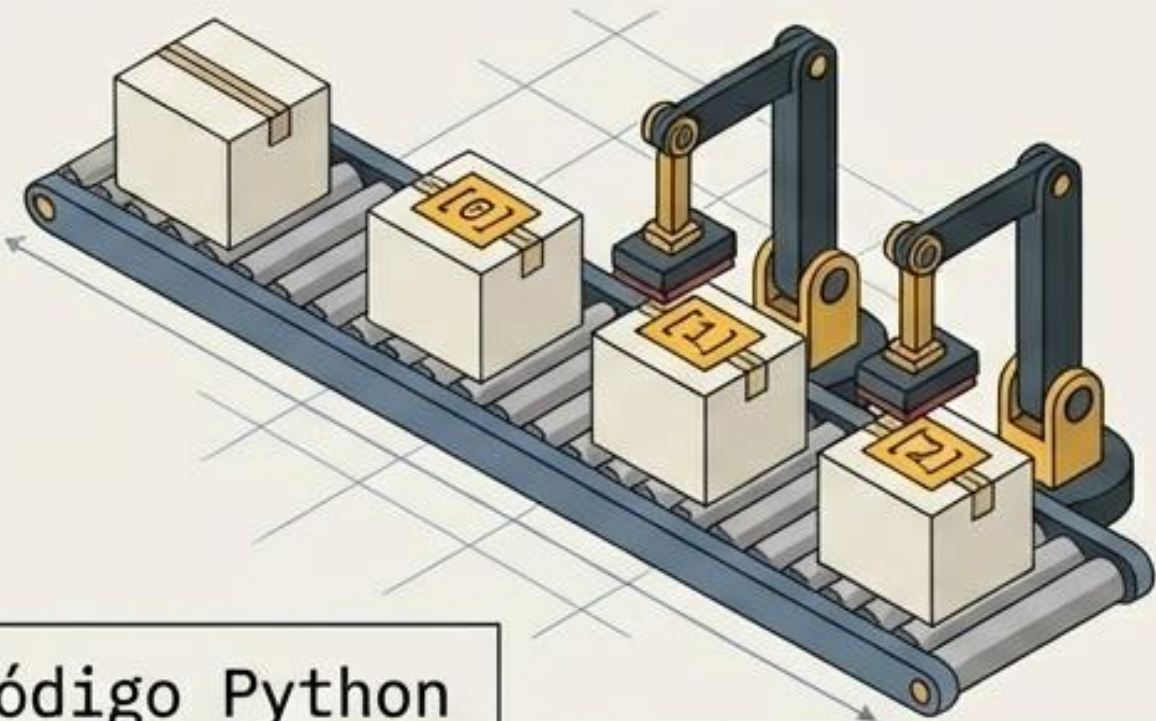
`range(start, stop, step)`



Superpoderes del Bucle for

Evita usar **range(len(lista))**. Utiliza estas herramientas nativas para escribir código más limpio, rápido y verdaderamente 'Pythónico'.

Upgrade Modules: **enumerate()**



Código Python

✗ `for i in range(len(tareas)):`

✓ `for index, tarea in enumerate(tareas):`

Upgrade Modules: **.items()**



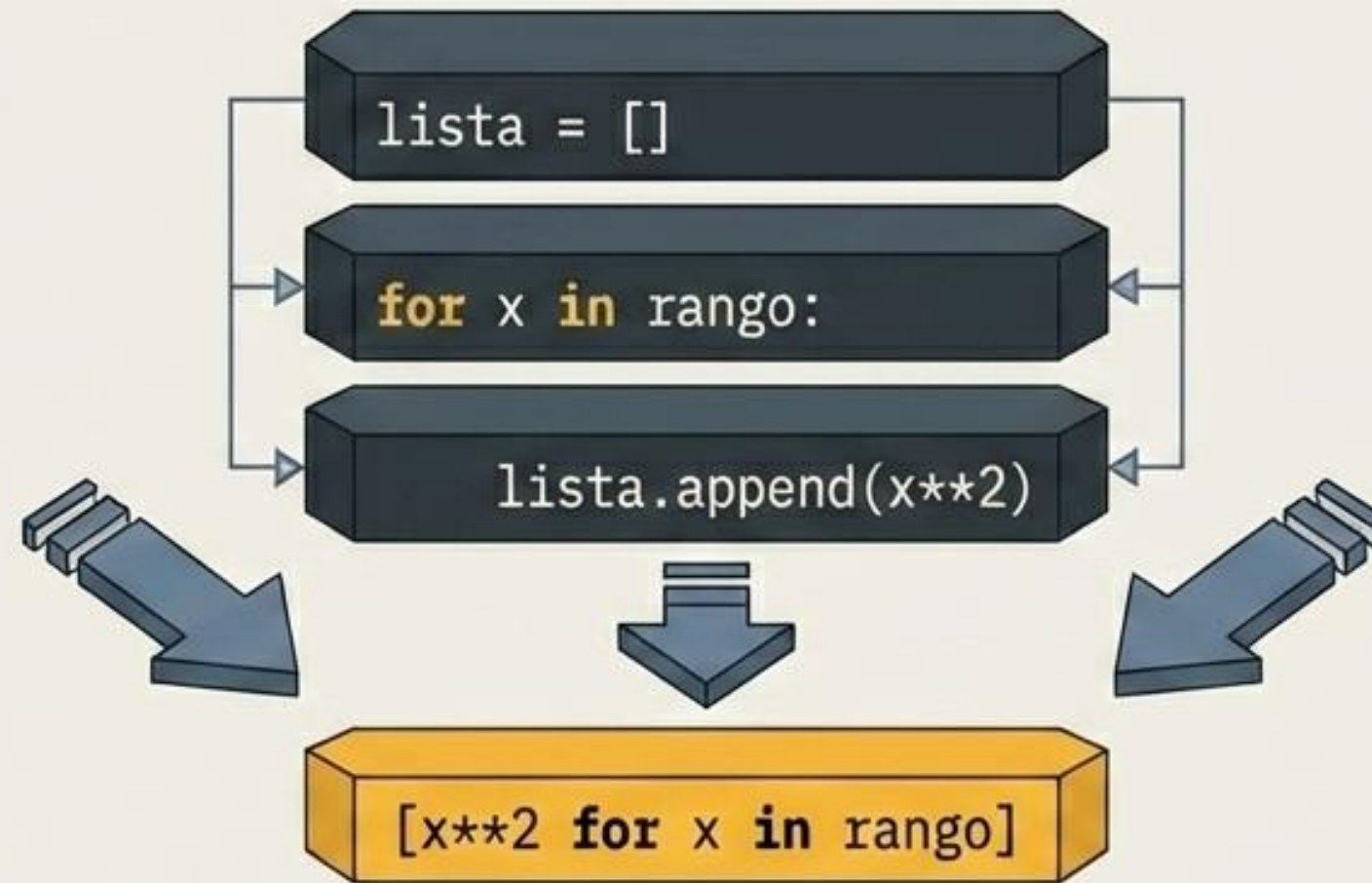
Código Python

✗ `for clave in inventario:`
 `valor = inventario[clave]`

✓ `for clave, valor in inventario.items():`

Magia en Una Línea: List Comprehensions

Las comprensiones de listas son una forma compacta y altamente optimizada para crear listas transformando otras secuencias.



[expresión (transformación) -> **for** elemento in iterable (bucle) -> **if** condición (filtro opcional)]

expresión (transformación) for elemento in iterable (bucle) if condición (filtro opcional)

El Bucle while: Condicionales Dinámicos

El bucle while brilla cuando la cantidad de iteraciones es impredecible. Continúa ejecutándose indefinidamente hasta que su condición de evaluación cambia de True a False.

Patrón: El Contador



Usado para rastrear cuántas veces ocurrió un evento o limitar intentos.

Patrón: El Acumulador



Usado para sumar totales progresivamente hasta alcanzar un límite.

Matriz de Decisión: ¿for o while?

Elegir la herramienta correcta define la calidad de tu código.

	Bucle for	Bucle while
Iteraciones conocidas (colecciones, rangos)	 Ideal / Perfecto	 No recomendado
Iteraciones desconocidas (basadas en eventos)	 Imposible / Difícil	 Ideal
Condiciones complejas (múltiples variables)	 Limitado	 Flexible (and / or)
Bucle infinito intencional (menús, servidores)	 No aplica	 Posible (while True)

Controladores de Tráfico

Gestiona el flujo interno de las iteraciones. En Python, el bloque `else` es único: se ejecuta únicamente si el bucle terminó de forma natural.

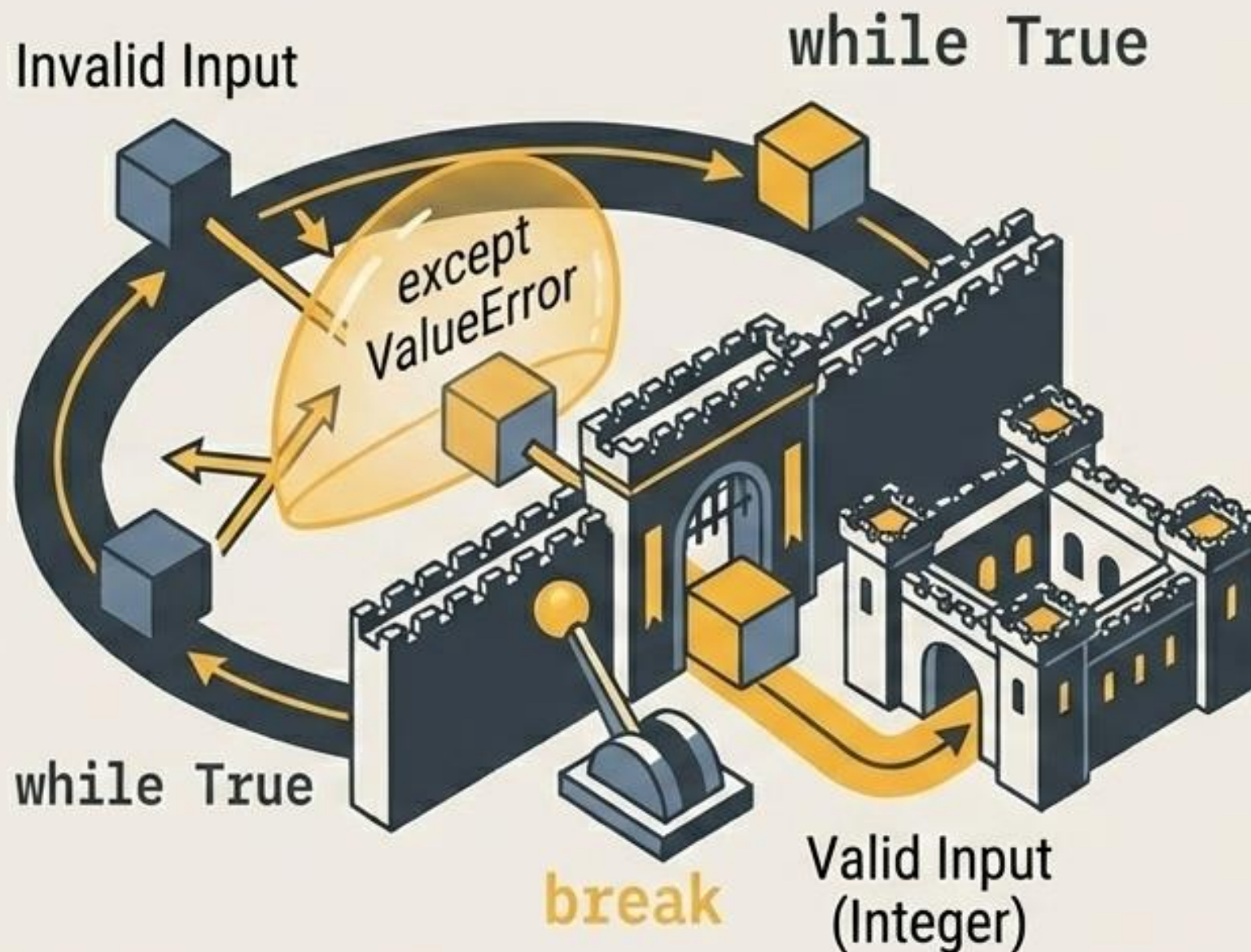
`break`

Destruye y sale del bucle por completo.



Robustez: Validación Inquebrantable

Nunca asumas que el usuario ingresará el dato correcto. Combina un bucle infinito con manejo de errores para forzar una entrada válida.



```
while True:  
    try:  
        numero = int(input('Ingresa número: '))  
        break  
    except ValueError:  
        print('Error: Inténtalo de nuevo')
```


Diagnóstico de Errores Comunes

Evita estas trampas clásicas que rompen silenciosamente la lógica de tu aplicación.

Anti-Patrón (Problema) 	Solución Pythónica 
Bucle infinito accidental. Olvidar actualizar la variable (ej. no incluir <code>counter += 1</code>).	Asegurar rigurosamente la progresión matemática hacia la condición de salida.
Modificar una lista mientras iteras sobre ella. (Causa saltos impredecibles y omisiones).	Iterar sobre una copia de la lista (<code>lista[:]</code>) o usar List Comprehensions para generar una nueva.
Índice fuera de rango en <code>while</code> . Usar <code><=</code> en lugar de <code><</code> contra la longitud (<code>len</code>).	Usar estrictamente <code>< len(lista)</code> o preferir directamente un bucle <code>for</code> .

Mejores Prácticas de Optimización

Escribir código que funcione es el primer paso. Escribir código eficiente y Pythónico es el arte de la maestría. Aplica estas reglas para reducir drásticamente los tiempos de ejecución.



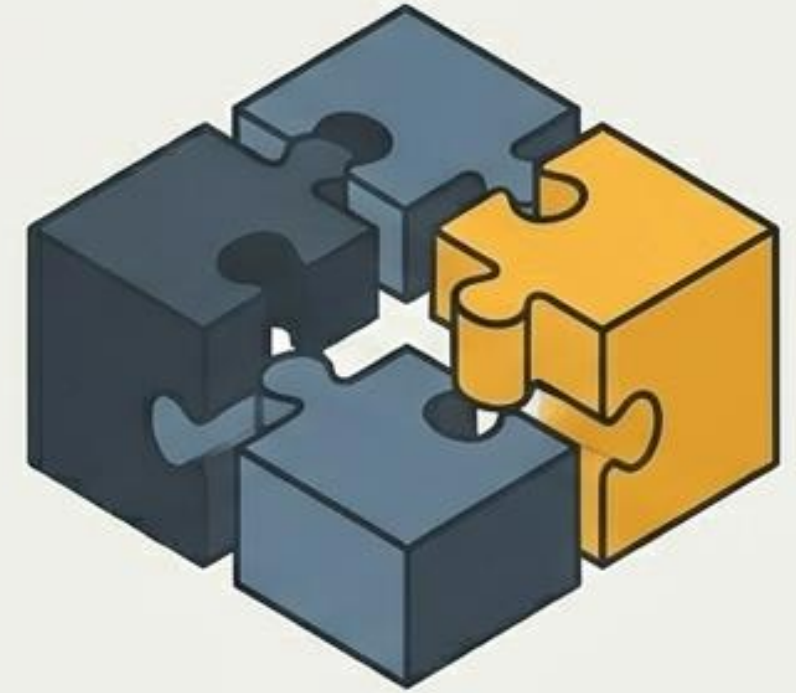
1. Saca el peso del bucle

Calcula longitudes con `len()` u operaciones estáticas pesadas antes de entrar al bucle, no durante cada iteración.



2. Poder Nativo

Reemplaza acumuladores manuales usando funciones construidas y optimizadas internamente como `sum()`, `max()`, `min()`.



3. Fusión de Strings

Evita concatenar texto con `' += '` dentro de un bucle. Acumula en una lista y fusiona todo al final usando `' '.join()`.